NewsBot Intelligence System 2.0

Reflective Journal

Trilok Kalani | ITAI 2373 Natural Language Processing | Final Project | Group: SOLO
GitHub: https://github.com/Tikskalani/ITAI2373-Portfolio

How I Organized Solo Work

Like the midterm, I did this project by myself, so organizing the work was less about splitting tasks and more about sequencing my own. I planned it in the order the system depends on itself. The refactor of the core pipeline had to come first, because the four new modules all lean on it. Only once the classifier, sentiment, and entity code were clean classes did I start the advanced modules, and I left the conversational layer and the web app for last because they sit on top of everything else. Working alone made those dependencies very concrete, since I felt every handoff myself. If this had been a real group, I would have split the modules by dependency, kept one shared repository, and used branches so two of us were not editing the same file, and the thing I would insist on up front is agreeing on the exact shape of the data passed between components, because that is what actually keeps a multi-part system from falling apart.

Individual Contributions

I built this project on my own, so every part of it is mine. The midterm version was one long notebook that worked but was hard to reuse. My main job on the final was to take that working pipeline apart and rebuild it as a real package, then add the four advanced modules on top. I designed the folder structure under src, decided what each class should own, and moved the preprocessing, TF-IDF features, classification, sentiment, and entity code out of the notebook into small files that each do one thing. On top of that base I wrote the new pieces: topic modeling with both LDA and NMF, an extractive summarizer with an optional transformer path, semantic search over the articles, a multilingual layer that detects and translates non-English text, and a conversational layer that reads a plain question and routes it to the right component. I also wrote the Flask web app so someone can use the whole thing in a browser, and the tests that check it all still works.

The piece I am most proud of is the conversational layer, because it ties everything together. Once each capability was its own class with a clear input and output, I could write an intent classifier that reads a question like "what is the sentiment?" and a query processor that hands it to the right module and formats the answer. That only worked because the refactor was done carefully first. It made me realize that most of the value in the final was not any one fancy model, it was the structure underneath that let the parts snap together.

Challenges Overcome

The hardest problems this time were about engineering, not algorithms. The first real headache was that I was editing files on Windows while running everything in a Linux sandbox, and the two did not always stay in sync. A file I had just changed would come back truncated when I read it from the other side. I lost

time before I understood what was happening, and I fixed it by doing all the final file writes in one place instead of bouncing between the two, and by rebuilding the notebook natively instead of copying it across. It was not glamorous, but it taught me that the environment itself is part of the problem you have to manage.

The second challenge was keeping the system light. Summarization and semantic search both have a heavy version that uses transformer models and a light version that does not. If I loaded the heavy models at import time, the whole thing became slow and the install ballooned. I ended up making the transformer paths lazy, so they only load if someone actually asks for them, and I made the defaults the light options that run on a plain CPU with no downloads. That way the app starts fast and still works out of the box, but the stronger models are there if you want them.

A third challenge carried over from the midterm and I made sure to keep the fix. Early on, short or out-of-scope text would get labeled as sport, because when none of the words were in the model's vocabulary it just fell back to the most common class. In the final I kept the uncertainty guard that returns 'uncertain' instead of guessing, and I checked that it still fires correctly after the refactor. Getting the classifier to admit when it does not know is the behavior I care about most, because a tool that is confidently wrong is worse than one that says it is not sure.

A smaller but annoying challenge was version control. At one point my push was rejected because the remote had moved ahead of my local copy, since I had also uploaded files through the GitHub website. I had to stop, pull and rebase, reapply my changes on top, and only then push. It was a good reminder that the repository is shared state even when you are the only person on it, and that rushing a push is how you lose work.

Lessons Learned

The biggest lesson was how much cleaner everything gets when you separate concerns properly. In the midterm the modules were tangled together in one notebook, and changing one thing risked breaking another. Once each capability was its own small class, I could test it alone, reuse it from the notebooks, the tests, and the web app, and reason about it without holding the whole system in my head. Writing the tests reinforced this, because a class that is hard to test is usually a class that is doing too much.

I also learned to respect the boundary between a light default and an optional upgrade. It is tempting to always reach for the biggest model, but a system that only runs on a powerful machine is not very useful. Designing so the plain version works everywhere and the strong version is one flag away felt like a professional decision rather than a homework one. And I learned again, from the multilingual work, that the honest approach is to translate to English and run the pipeline I trust, rather than pretending my English-trained models understand every language directly.

Future Improvements

If I keep building on this, the first change is still the sentiment model. The lexicon approach misreads financial language and will score a profit warning as positive, and that is the weakest part of the system. I would replace it with a finance-tuned model like FinBERT. After that I would fine-tune the entity recognizer on recent news so it stops mislabeling modern companies and places, and I would add topic

modeling that runs over time so the system can show how themes rise and fall instead of giving one static snapshot. On the product side, I would deploy the web app to a public URL so it has a live demo link, add batch upload so a user can analyze a whole feed at once, and cache results so repeated queries are instant. Longer term, the interesting extensions are bias detection and simple fact-checking, since those move the system from describing the news to actually helping a reader judge it.

Overall, this project pushed me to think about a whole system instead of a single model. I had to manage a real environment, keep the thing fast, design for reuse, test it, document it, and be honest about where it falls short. Those habits feel much closer to real engineering than a normal assignment does, and they are the part I will carry into the next project.

Professional Development Impact

More than any single technique, this project taught me to build and defend a whole system. Taking a working notebook and turning it into a documented, tested package is a different skill from getting a model to train, and it is the one that maps most directly to real work. I had to make design decisions and live with them, write things down so someone else could follow, and keep the system honest about its limits instead of overselling it. I also practiced explaining the same work two ways, once in technical detail for the documentation and once in plain business terms for the executive summary, which is a skill I expect to use constantly. By the end, the project felt less like an assignment and more like a small product I would be comfortable showing in an interview and talking through in depth.